Uniwersytet Mikołaja Kopernika w Toruniu
Wydział Matematyki i Informatyki

Maksymilian Kudelski
numer albumu: 308123
Kierunek Informatyka

Praca inżynierska

Aplikacja CRISP-DM

Opiekun pracy dyplomowej
dr Jerzy Białkowski

Toruń 2026

Spis treści

Wstęp **5**

1 Przedstawienie przedmiotu pracy **7**

2 Założenia projektowe **9**
2.1 Wymagania funkcjonalne 9
2.1.1 Funkcjonalności obowiązkowe 9
2.1.2 Funkcjonalności pożądane 10
2.2 Wymagania niefunkcjonalne 10
2.2.1 Wymagania obowiązkowe 10
2.2.2 Wymagania pożądane 11

3 Opis narzędzi i technologii **13**
3.1 Języki programowania 13
3.1.1 Python 13
3.1.2 JavaScript 14
3.2 Baza danych 14
3.3 Backend 14
3.4 Frontend 15
3.5 Konteneryzacja 16

4 Struktura części programistycznej **17**
4.1 Struktura stworzonego systemu 18
4.2 Budowa systemu 18
4.3 Moduł 1 18
4.4 Moduł 2 18
4.5 Moduł 3 18
4.6 Baza danych 18

5 Opis wybranych szczegółów implementacji **19**

6 Procedura testowa i testy **21**

7 Opis instalacji i używania **23**

8 Podsumowanie **25**

Bibliografia 32

Wstęp

W tym miejscu proszę zamieścić wstęp.

Tekst do wpisania w pojedynczym akapicie może być łamany w dowolnych miejscach, byle nie był rozdzielany pustymi liniami. Przypominam, że pusta linia sugeruje podział i rozpoczęcie nowego akapitu. Ewentualne łamania linii możemy wymuszać komendą `\linebreak`, a zakończenie linii przez „\\" lub komendą `\newline`. Dodatkowo możemy wymuszać łamanie strony komendą `\pagebreak`, a zakończenie strony przez komendą `\newline` (ale bardzo bym prosił nie nadużywać tych komend).

Przypominam też zalecenia/wytyczne dotyczące zawartości wstępu – z zachowaniem numeracji z oryginalnego dokumentu.

9. Wstęp do pracy powinien zawierać informacje jaki jest cel / teza pracy.

10. Wstęp pracy powinien zawierać informację o tym jaki jest wkład własny studentki/studenta. Przykładowo: informację, czy wykorzystany algorytm

    (a) został wymyślony przez studentkę/studenta,

    (b) jest twórczą modyfikacją/kompilacją znanych algorytmów,

    (c) jest własną implementacją znanych algorytmów,

    (d) jest funkcją biblioteczną lub został w całości skopiowany z cytowanego źródła.

Zwracam uwagę, że przy *jednostronnym* układzie pracy każdy nowy rozdział powinien rozpoczynać **od nowej strony**, a przy *dwustronnym* układzie pracy każdy nowy rozdział powinien rozpoczynać się **od nowej kartki** (czyli od strony nieparzystej).

Rozdział 1

Przedstawienie przedmiotu pracy

W większości prac warto będzie w tym miejscu napisać rozdział poświęcony **teoretycznej** stronie zagadnienia.

W szczególnych przypadkach zagadnienia będą tak obszerne, że warto będzie je podzielić na kilka rozdziałów. Na chwilę obecną przewiduję, że będzie tak u panów Doriana i Maksymiliana.

Przykład jakiejś tabelki ilustrującej coś..

Tabela 1.1: Przykładowa tabela

liczba	x	$x^2 = x \cdot x$
jeden	1	1
dziesięć	10	100
dwadzieścia	20	400

(być może później dopiszę więcej...)

Rozdział 2

Założenia projektowe

Na podstawie przeprowadzonej analizy określono wymagania funkcjonalne oraz niefunkcjonalne dla projektowanego systemu. Dodatkowo wymagania podzielono na obowiązkowe oraz pożądane, co pozwala wyznaczyć minimalny zakres implementacji oraz funkcjonalności rozszerzające możliwości aplikacji.

2.1 Wymagania funkcjonalne

2.1.1 Funkcjonalności obowiązkowe

- rejestracja i logowanie użytkowników,
- zarządzanie kontem użytkownika,
- tworzenie, edycja i usuwanie projektów analitycznych,
- przechowywanie projektów przypisanych do użytkownika,
- import plików CSV, XLSX, XLS oraz ODS,
- walidacja poprawności przesyłanych plików,
- podgląd zawartości zaimportowanych danych,
- odczyt nazw kolumn, typów danych oraz brakujących wartości,
- czyszczenie danych poprzez usuwanie pustych rekordów i duplikatów,
- identyfikacja i uzupełnianie brakujących danych,

- obsługa wielu arkuszy w plikach Excel i ODS,
- generowanie podstawowych statystyk danych,
- wizualizacja danych w postaci wykresów,
- filtrowanie i transformacja danych,
- podział danych na zbiory treningowe, testowe oraz walidacyjne,
- wybór algorytmu uczenia maszynowego (opcje podstawowe np. regresja, las drzerw losowych),
- trenowanie modeli uczenia maszynowego,
- ewaluacja modeli z wykorzystaniem metryk jakości,
- generowanie raportów PDF zawierających wyniki analiz.

2.1.2 Funkcjonalności pożądane

- porównywanie wyników różnych modeli,
- wybór algorytmu uczenia maszynowego (dodatkowe opcje np. gradient boosting),
- konfiguracja parametrów modelu,
- pobieranie wygenerowanych raportów PDF,
- archiwizacja raportów i historii analiz.

2.2 Wymagania niefunkcjonalne

2.2.1 Wymagania obowiązkowe

- aplikacja musi działać w architekturze klient–serwer,
- backend musi zostać wykonany z wykorzystaniem frameworka Django,
- frontend musi zostać wykonany z wykorzystaniem biblioteki React,

- interfejs użytkownika musi wykorzystywać Tailwind CSS,
- dane muszą być przechowywane w bazie PostgreSQL,
- aplikacja musi umożliwiać realizację etapów metodologii CRISP-DM od Data Understanding do Evaluation,
- aplikacja musi zabezpieczać aplikację przed SQL Injection, XSS oraz CSRF,
- aplikacja musi walidować przesyłane pliki pod względem formatu i typu MIME,
- aplikacja musi zapewniać integralność danych zapisanych w bazie,
- aplikacja musi być odporna na błędne dane wejściowe.

2.2.2 Wymagania pożądane

- aplikacja powinna umożliwiać dalszą rozbudowę o kolejne etapy metodologii CRISP-DM (Business Understanding oraz Deployment),
- interfejs użytkownika powinien umożliwiać płynne przechodzenie między etapami analizy oraz łatwy powrót do wcześniejszych kroków i wprowadzanie korekt.
- interfejs użytkownika powinien być intuicyjny i responsywny,
- aplikacja powinna obsługiwać pliki o rozmiarze co najmniej 50 MB,
- aplikacja powinna prezentować czytelne komunikaty błędów.

Rozdział 3

Opis narzędzi i technologii

W procesie projektowania i implementacji aplikacji wykorzystano nowoczesny stos technologiczny umożliwiający budowę skalowalnego systemu webowego zgodnego z aktualnymi standardami tworzenia oprogramowania.

3.1 Języki programowania

3.1.1 Python

Jako język programowania części pracy związanej z realizacją etapów metodologii CRISP-DM oraz backendu wybrano Pythona w wersji 3.12.

Python jest wysokopoziomowym, interpretowanym językiem programowania ogólnego przeznaczenia. Charakteryzuje się prostą i czytelną składnią, dzięki czemu znajduje zastosowanie zarówno w tworzeniu aplikacji webowych, jak i analizie danych, automatyzacji procesów, obliczeniach naukowych oraz projektach związanych ze sztuczną inteligencją i uczeniem maszynowym. Ze względu na swoją uniwersalność oraz bogaty ekosystem bibliotek jest obecnie jednym z najpopularniejszych języków programowania na świecie.

Posiada on rozbudowany ekosystem bibliotek do obróbki danych i uczenia maszynowego (pandas, scipy, scikit-learn) oraz budowy backendów aplikacji webowych (Django). Umożliwia to ograniczenie liczby wykorzystywanych języków programowania w warstwie serwerowej poprzez wyeliminowanie konieczności wywoływania kodu napisanego w języku R. Wybór Pythona pozwolił również na łatwą integrację modułów odpowiedzialnych za przetwarzanie danych, trenowanie modeli oraz udostępnianie wyników za pomocą

interfejsu webowego.

3.1.2 JavaScript

Jako język programowania warstwy frontendowej aplikacji wybrano JavaScript. W projekcie wykorzystano środowisko uruchomieniowe Node.js w wersji 24.14.0 oraz bibliotekę React w wersji 19.

JavaScript jest wysokopoziomowym, interpretowanym językiem programowania wykorzystywanym przede wszystkim do tworzenia interaktywnych aplikacji internetowych. Stanowi podstawową technologię współczesnych aplikacji webowych obok języków HTML i CSS. Dzięki możliwości uruchamiania zarówno po stronie przeglądarki, jak i serwera znajduje zastosowanie w budowie interfejsów użytkownika, aplikacji sieciowych oraz systemów czasu rzeczywistego. Język ten stanowi podstawę działania biblioteki React, która została wykorzystana do budowy interfejsu użytkownika aplikacji.

Zastosowanie JavaScriptu pozwoliło na stworzenie dynamicznego i responsywnego interfejsu komunikującego się z backendem za pomocą interfejsu REST API. Wykorzystanie środowiska Node.js umożliwiło zarządzanie zależnościami projektu oraz automatyzację procesu budowania aplikacji. Bogaty ekosystem bibliotek i narzędzi wspierających rozwój aplikacji frontendowych znacząco przyspiesza proces implementacji oraz utrzymania projektu. Dodatkową zaletą jest duża popularność języka, przekładająca się na szeroką dostępność dokumentacji, materiałów edukacyjnych oraz wsparcia społeczności programistycznej.

3.2 Baza danych

Jako system zarządzania relacyjną bazą danych wykorzystano PostgreSQL w wersji 18.3. PostgreSQL jest jednym z najpopularniejszych systemów bazodanowych typu open source, zapewniającym wysoką wydajność, niezawodność oraz zgodność ze standardem SQL. Został on wykorzystany do przechowywania danych aplikacji oraz realizacji operacji związanych z ich przetwarzaniem.

3.3 Backend

Warstwa serwerowa została zaimplementowana w języku Python z wykorzystaniem frameworka Django 6.0.3. Django umożliwia szybkie tworzenie

bezpiecznych aplikacji internetowych zgodnie z architekturą Model–View–Template (MVT). Dodatkowo wykorzystano framework Django REST Framework służący do budowy interfejsu API. W projekcie zastosowano następujące biblioteki:

- asgiref 3.11.1 – obsługa standardu ASGI,
- Django 6.0.3 – główny framework aplikacji,
- sqlparse 0.5.5 – parser składni SQL wykorzystywany przez Django,
- Django REST Framework – implementacja usług REST API,
- psycopg2-binary – komunikacja z bazą PostgreSQL,
- django-filter – filtrowanie danych zwracanych przez API,
- python-decouple – zarządzanie zmiennymi środowiskowymi,
- dj-database-url – konfiguracja połączeń bazodanowych,
- django-simple-history – przechowywanie historii zmian danych,
- gunicorn – serwer aplikacyjny dla środowiska produkcyjnego,
- pytest – framework testowy,
- pytest-django – rozszerzenie testów dla aplikacji Django.

3.4 Frontend

Warstwa kliencka została zaimplementowana przy użyciu środowiska Node.js w wersji 24.14.0 oraz biblioteki React 19, będącej najnowszą stabilną wersją kompatybilną z wykorzystanym środowiskiem uruchomieniowym. React umożliwia tworzenie dynamicznych i responsywnych interfejsów użytkownika opartych na architekturze komponentowej. Do stylizacji interfejsu wykorzystano framework Tailwind CSS. Narzędzie to pozwala na szybkie tworzenie nowoczesnych i spójnych wizualnie interfejsów użytkownika przy użyciu zestawu gotowych klas narzędziowych.

3.5 Konteneryzacja

W celu ułatwienia wdrażania oraz zapewnienia powtarzalności środowiska uruchomieniowego zastosowano platformę Docker. Konteneryzacja pozwala na izolację poszczególnych komponentów systemu, takich jak aplikacja backendowa, frontend oraz baza danych, co upraszcza proces konfiguracji, testowania i wdrażania aplikacji na różnych środowiskach.

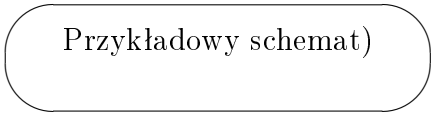
Rozdział 4

Struktura części programistycznej

Tutaj oczekiwałbym przedstawienia „całościowej" projektu programistycznego.

Zasadniczo można by ten rozdział podzielić na część poświęconą stronie sprzętowej (urządzenia i połączenia pomiędzy nimi) oraz programistycznej (moduły (w tym programy, biblioteki, bazy danych) i zależności pomiędzy nimi). Oczywiście poza opisem słownym warto byłoby zilustrować te opisy stosownymi ilustracjami (np. diagramem wdrożenia dla strony „sprzętowej" i komponentów dla strony „programistycznej"). Oczywiście, **trzeba przedstawić cały system, a jeśli praca dotyczy tylko wybranych jego części, to trzeba wyraźnie zaznaczyć których**.

W szczególnych przypadkach może okazać się zasadnym podzielenie tego rozdziału na dwa lub więcej rozdziałów. Jednak zazwyczaj wystarczy podział na podrozdziały. Dwa pierwsze z nich opisywałyby wówczas „całościową" strukturę projektu odpowiednio od strony sprzętowej i programistycznej. Kolejne podrozdziały mogłyby opisywać te moduły, które stworzył autor (względnie wybrane z nich) Osobny podrozdział warto byłoby poświęcić strukturze bazy danych.



Rysunek 4.1: Diagram wdrożenia (podpis pod rysunkiem)

Rysunek 4.2: Podpis pod drugim rysunkiem

4.1 Struktura stworzonego systemu

4.2 Budowa systemu

4.3 Moduł 1

4.4 Moduł 2

4.5 Moduł 3

4.6 Baza danych

Rozdział 5

Opis wybranych szczegółów implementacji

Listing 5.1: Listing który można łamać pomiędzy stronami

```
for (int i = 0; i < 10; i++)
{
    Console.WriteLine("Linia {0}", i); // komentarz
}
```

Listing 5.2: Opis listingu

```
string[] tekst = {"Tekst", "do", "wypisania"};

foreach (string wyraz in tekst)
{
    Console.WriteLine(wyraz);
}
```

Kod źródłowy 5.1: To jest właściwy opis listingu

Rozdział 6

Procedura testowa i testy

Rozdział 7

Opis instalacji i używania

Pod koniec warto przedstawić opis procedury instalacji/wdrażania systemu oraz „podręcznik użytkownika”.

Rozdział 8

Podsumowanie

Na koniec proszę umieścić podsumowanie.

Warto zamieścić w nim m.in. możliwe kierunki i ścieżki rozwoju Państwa systemu.

Odesłania do listingu 5.1 oraz rysunku 4.2. Cytowanie literatury z pozycji [AA1, Twierdzenie 1.2].

Spis rysunków

4.1 Diagram wdrożenia (podpis pod rysunkiem) 18
4.2 Podpis pod drugim rysunkiem 18

Spis tabel

1.1 Przykładowa tabela 7

Lista kodów źródłowych

5.1 Krótka wersja podpisu 19

Bibliografia

[AA1] Autor Anonimowy, ...